KSFUPRO1KU, Functional Programming ITU, Spring 2026

Exercise 6

Last update February 17, 2026

This exercise sheet must be handed in via LearnIt.

You solve the assignments in your groups.

Your name must be part of the filename, e.g., `FP-05-<name>-<name>.fsx`.

An example: `FP-05-MadsAndersen-HugoHansen.fsx`.

You can only upload one file and it must be of type `fs` or `fsx`.

It is important that you annotate your own code with comments. It is also important that you apply a functional style, i.e., no loops and no mutable variables.

For this hand–in you also need to consider scenarios where your solutions should return an error, i.e., an exception. The requirement is, that no matter what input you pass to your function that fulfils the function type, then the function should return the intended answer or an exception. It is up to you to define the exceptions and whether they should carry extra information, like error messages.

The exercises 6.1 to 6.2 refer to code defined in the slide deck from lecture 5 about expression trees.

Exercise 6.1 Complete the program skeleton for the interpreter presented in the slide deck from lecture 5 on expression trees.
The declaration for the abstract syntax for *arithmetic expressions* follows the grammar:

```
type aExp =             (* Arithmetical expressions *)
  | N of int            (* numbers *)
  | V of string         (* variables *)
  | Add of aExp * aExp  (* addition *)
  | Mul of aExp * aExp  (* multiplication *)
  | Sub of aExp * aExp  (* subtraction *)
```

The declaration of the abstract syntax for *boolean expressions* is defined as follows.

```
type bExp =              (* Boolean expressions *)
  | TT                   (* true *)
  | FF                   (* false *)
  | Eq of aExp * aExp    (* equality *)
  | Lt of aExp * aExp    (* less than *)
  | Neg of bExp          (* negation *)
  | Con of bExp * bExp   (* conjunction *)
```

The conjunction of two boolean values returns true if both values are true.
The abstract syntax for the statements are defined as below:

```
type stm =                      (* statements *)
  | Ass of string * aExp        (* assignment *)
  | Skip
  | Seq of stm * stm            (* sequential composition *)
  | ITE of bExp * stm * stm     (* if-then-else *)
  | While of bExp * stm         (* while *)
```

Define 5 examples and evaluate them.

For instance, consider the example `stmt0` and initial state `state0` below.

```
let stmt0 = Ass("res",(Add(N 10, N 30)))
let state0 = Map.empty
```

You can then run the example as follows

```
> I stmt0 state0;;
val it : Map<string,int> = map [("res", 40)]
```

and get the result state with variable `res` assigned the value 40 (as expected).

Exercise 6.2 Extend the abstract syntax and the interpreter with *if-then* and *repeat-until* statements.

```
let rec I stm s = match stm with
    | Ass(x,a) -> update x ( ... ) s
    | Skip -> ...
    | Seq(stm1, stm2) -> ...
    | ITE(b,stm1,stm2) -> ...
    | While(b, stm) -> ...
    | RU(b, stm) -> ...  (* Repeat Until *)
    | IT (b,stm1) -> ... (* If Then *)
```

Again we refer to the slide deck from the lecture 5.

Hint: The if–then statement is similar to the if–then–else statement when you do a `Skip` statement in the else branch.

Hint: The slide deck from lecture 5 provides an example of running the `fac` example program with a state mapping x to 4.

Exercise 6.3 Suppose that an *expression* of the form $inc(x)$ is added to the abstract syntax. It adds one to the value of x in the current state, and the value of the expression is this new value of x. The expression $inc(x)$ should be added to the type `aExp`.

How would you refine the interpreter to cope with this construct?

Again we refer to the slide deck from the lecture 5

Hint: Adding $inc(x)$ to `aExp`, means that evaluating an expression may also update the state. Hense the state must be returned which has a rippling effect on the evaluation functions.

This task is only to describe how you would solve the task. There is no code to hand–in.